Spec2Test Automated Workflow: Input & Output Brief

Automated HIL Test Case Preparation, Signal Validation & Requirement Traceability — Automotive Headlamp Control System (BCM)

Executive Overview: In automotive software validation, engineers routinely translate dozens of system-level requirements into structured Hardware-in-the-Loop (HIL) test cases. The **Spec2Test workflow** automates this preparation and initial validation phase. Given raw engineering requirements and a valid vehicle signal dictionary, the workflow extracts positive, negative, and boundary test scenarios, runs rule-based validation to verify coverage and signal compliance, fills detected scenario gaps, and delivers a ready-to-review test package in JSON, CSV, and Markdown formats.

1. Input Provided to the Workflow: Requirements & Signal Allowlist

To initiate test generation, an engineer sends a structured JSON payload to the **n8n Webhook**. Using an **Automatic Headlamp Control System** managed by the Body Control Module (BCM) as a representative example, the input specifies how low beam headlights must respond under varying ambient lighting conditions and sensor fault states, alongside the exact signal identifiers permitted in test scripts.

Sample Webhook Input Payload (JSON)

```
{
  "document_text": "REQ-BCM-001: When ignition is ON and ambient light is below 400 lux continuously for more than 2 seconds, the BCM shall switch ON the low beam headlights within 300 milliseconds.\n\nREQ-BCM-002: When ignition is ON and ambient light is above 600 lux continuously for more than 5 seconds, the BCM shall switch OFF the low beam headlights within 500 milliseconds.\n\nREQ-BCM-003: If the ambient light sensor signal is invalid, the BCM shall store DTC B1234 and activate the low beam headlights as a safety measure.",
  "signal_dictionary": [
    "IgnitionStatus",
    "AmbientLight",
    "AmbientLightSensorStatus",
    "LowBeamCommand",
    "DTC_B1234"
  ]
}
```

Input Payload Schema & Permitted Signal Dictionary

Constraining the workflow to a predefined `signal_dictionary` prevents the use of unverified or non-existent CAN/LIN bus signals during test preparation.

Input Field / Signal Name	Category	Engineering Purpose & Description
`document_text`	Payload Field	Plain-text automotive system requirements authored by systems engineers (e.g., `REQ-BCM-001` through `REQ-BCM-003`).
`signal_dictionary`	Payload Field	Strict allowlist of valid system and bus signals that generated HIL test cases are permitted to reference.
`IgnitionStatus`	Input Signal	Vehicle ignition state indicator (e.g., `ON`, `OFF`).
`AmbientLight`	Input Signal	Ambient light sensor reading measured in lux (thresholds at `400 lux` and `600 lux`).
`AmbientLightSensorStatus`	Input Signal	Diagnostic validity status of the ambient light sensor signal (`VALID` or `INVALID`).
`LowBeamCommand`	Output Signal	BCM actuator command controlling low beam headlamp state (`ON` or `OFF`).
`DTC_B1234`	Diagnostic Code	Diagnostic Trouble Code stored in BCM fault memory when the ambient light sensor signal is invalid.

2. End-to-End n8n Workflow Execution Pipeline

Once the JSON payload reaches the n8n endpoint, an eight-stage automated pipeline parses the text, synthesizes test scenarios, validates engineering rules, resolves coverage gaps, and compiles the final traceability report.

Stage	n8n Workflow Node	Automated Processing Responsibility
01	**Webhook Receives Input**	Receives the HTTP POST payload containing the requirement text and allowed signal dictionary.
02	**Prepare Input**	Sanitizes formatting, normalizes line breaks, and prepares the payload for requirement segmentation.
03	**Split Requirements**	Parses the document into three individual requirement records (`REQ-BCM-001`, `REQ-BCM-002`, `REQ-BCM-003`) for granular tracking.
04	**Extract Test Cases**	Evaluates each requirement's preconditions, thresholds, durations, and timing rules to generate candidate positive, negative, and boundary test cases.

Stage	n8n Workflow Node	Automated Processing Responsibility
05	**Analyze (Validation Logic)**	Executes JavaScript verification rules to check requirement coverage, expected outcomes, boundary adherence, missing test types, invalid signal usage, and duplicate cases.
06	**Gaps to Fill?**	Evaluates validation results; if coverage gaps or missing boundary scenarios are detected, triggers a supplemental extraction pass to generate additional test cases.
07	**Finalize Report**	Consolidates all verified test scenarios, computes coverage and quality metrics, and generates the requirement-to-test traceability matrix.
08	**Respond to Webhook**	Returns the final structured JSON response containing test cases, validation metrics, CSV export data, and a formatted Markdown report.

3. Generated Output: Draft HIL Test Cases by Requirement

Below are the draft HIL test scenarios produced by the workflow across the three BCM headlamp requirements. Each requirement is decomposed into testable stimulus parameters and measurable expected responses.

Requirement REQ-BCM-001: Low Beam Activation Below 400 Lux

Original Requirement: When ignition is ON and ambient light is below 400 lux continuously for more than 2 seconds, the BCM shall switch ON the low beam headlights within 300 milliseconds.

Engineering Boundary Insight: Because the specification states strictly **below 400 lux** (not 400 lux or below), `TC-003` explicitly verifies that holding ambient light at exactly `400 lux` does not trigger headlamp activation.

Test ID	Scenario Type	Input Stimulus Parameters	Expected BCM Output & Timing

TC-001	Normal Operation (Positive)	`IgnitionStatus = ON`
		`AmbientLight = 300 lux`
		`Duration = 2.5 seconds`
			`LowBeamCommand = ON`
			Response time: Within 300 ms

TC-002	Light Above Threshold (Negative)	`IgnitionStatus = ON`
		`AmbientLight = 500 lux`
		`Duration = 3.0 seconds`
			`LowBeamCommand` remains `OFF`

Test ID	Scenario Type	Input Stimulus Parameters	Expected BCM Output & Timing

TC-003	Exact Threshold (Boundary)	`IgnitionStatus = ON`
		`AmbientLight = 400 lux`
		`Duration = 3.0 seconds`
			No activation (strictly enforces the stated below-400-lux condition)

Requirement REQ-BCM-002: Low Beam Deactivation Above 600 Lux

Original Requirement: When ignition is ON and ambient light is above 600 lux continuously for more than 5 seconds, the BCM shall switch OFF the low beam headlights within 500 milliseconds.

Test ID	Scenario Type	Input Condition	Expected BCM Result
TC-004	Normal Deactivation (Positive)	`AmbientLight = 700 lux` continuously for `6 seconds` (with `IgnitionStatus = ON`)	Headlights switch `OFF` (`LowBeamCommand = OFF`) within 500 ms
TC-005	Light Below Deactivation Threshold (Negative)	`AmbientLight = 500 lux` continuously for `6 seconds`	Headlights remain `ON` (`LowBeamCommand = ON`)
TC-006	Exact Threshold (Boundary)	`AmbientLight = 600 lux` continuously for `6 seconds`	No `OFF` command issued (enforces the stated above-600-lux condition)

Requirement REQ-BCM-003: Invalid Sensor Signal & Fail-Safe Activation

Original Requirement: If the ambient light sensor signal is invalid, the BCM shall store DTC B1234 and activate the low beam headlights as a safety measure.

Test ID	Scenario Type	Input Condition	Expected BCM Result
TC-007	Fault Memory Storage (Positive)	`AmbientLightSensorStatus = INVALID`	Diagnostic Trouble Code `DTC_B1234` is stored in BCM memory
TC-008	Fail-Safe Headlamp Activation (Positive)	`AmbientLightSensorStatus = INVALID`	Fail-safe state engaged: `LowBeamCommand = ON`
TC-009	Dynamic Fault Transition (State Change)	`AmbientLightSensorStatus` transitions from `VALID` to `INVALID`	Safety action is immediately triggered (low beams ON and fault logged)

Engineering Verification Scope: These generated scenarios serve as structured draft HIL test cases to accelerate test authoring. Test engineers review these outputs to confirm bench-specific signal timing and fault recovery criteria prior to HIL rig execution.

4. Final Report: Traceability Matrix, Coverage Metrics & Gap Analysis

The workflow's **Finalize Report** node compiles execution statistics, maps every generated test case back to its parent requirement, and highlights specification ambiguities that require engineering clarification.

Workflow Validation & Traceability Summary

Across the sample run, **3 system requirements** were processed into **9 structured test cases**, achieving **100% requirement coverage** (every input requirement mapped to positive, negative, or fault-handling verification cases) with **0 duplicate test cases** detected by the JavaScript analyzer.

Requirement ID	Requirement Focus Area	Mapped HIL Test Cases	Case Count
REQ-BCM-001	Low beam activation (< 400 lux for > 2 s, ≤ 300 ms response)	`TC-001`, `TC-002`, `TC-003`	3
REQ-BCM-002	Low beam deactivation (> 600 lux for > 5 s, ≤ 500 ms response)	`TC-004`, `TC-005`, `TC-006`	3
REQ-BCM-003	Invalid ambient light sensor fault (DTC B1234 & fail-safe ON)	`TC-007`, `TC-008`, `TC-009`	3
Total / Summary	**100% Requirement Traceability Coverage*** (0 Duplicates)	**TC-001 through TC-009**	**9**

**Note: Requirement coverage confirms that all input requirements have corresponding validation scenarios; engineers still review edge-case combinations during formal test sign-off.*

Detected Specification Gaps Flagged for Engineering Review

Beyond generating test cases, the analysis stage identifies unstated behaviors and edge conditions in the original requirements so systems and test engineers can resolve ambiguities early:

- **Sensor Recovery Behavior:** What should happen to `LowBeamCommand` when `AmbientLightSensorStatus` transitions from `INVALID` back to `VALID`?
- **Ignition-Off Priority:** Should the low beam headlights switch `OFF` immediately when `IgnitionStatus` changes to `OFF`, regardless of ambient lux timers?
- **Fault Detection Timing:** What is the maximum permitted response time (debounce and actuation window) for detecting an invalid sensor signal under `REQ-BCM-003`?

- **Diagnostic Recovery Procedure:** Is there a defined healing or clearing procedure for `DTC_B1234` across ignition cycles once sensor validity is restored?

5. Complete Deliverable Package Returned by the Workflow

When the workflow completes execution, the **Respond to Webhook** node returns a comprehensive JSON response designed for direct integration with test management tools and engineering review pipelines:

- **Structured Draft Test Cases:** Complete stimulus parameters, signal names, durations, and expected BCM responses (`TC-001` through `TC-009`).
- **Requirement IDs & Traceability Matrix:** Bidirectional mapping linking each system requirement (`REQ-BCM-001` to `REQ-BCM-003`) to its verification test cases.
- **Coverage & Quality Metrics:** Automated calculation of requirement coverage percentage, total scenarios generated, and validation pass rates.
- **Detected Gaps & Ambiguities:** Actionable engineering review questions highlighting missing timing constraints, state transitions, or recovery rules.
- **Duplicate-Case Analysis:** Verification logs confirming redundant test scenarios were filtered out (`0` duplicates).
- **Extraction & Validation Diagnostics:** Structured logs capturing any parsing issues or unmapped signal references.
- **CSV-Formatted Test Case Data:** Ready-to-import tabular data for spreadsheet tracking or ALM/test management import.
- **Formatted Markdown Report:** Human-readable executive summary and test specification report for rapid team review.

Operational Scope & Engineering Sign-Off: This workflow automates test case preparation, signal validation, and requirement gap analysis—significantly reducing manual test authoring time across large requirement sets. It does not execute tests directly on physical HIL bench hardware; test engineers review, refine, and approve the generated draft cases before bench execution.